

Homework 2

Week 3

1、 $T(n) = n + \sum_{i=1}^n T(i)$, $T(1) = 1$, 求 $T(n)$ 的非递推解析式

解:

$$\begin{aligned} T(n+1) &= n+1 + \sum_{i=1}^n T(i) \\ &= T(n) + 1 + n + \sum_{i=1}^{n-1} T(i) \\ &= 2T(n) + 1 \end{aligned}$$

$\therefore T(n+1) + 1 = 2(T(n) + 1)$, $\{T(n) + 1\}$ 是首项为 $T(1) + 1 = 2$, 公比为 2 的等比数列
 $\therefore T(n) + 1 = 2^n$, $T(n) = 2^n - 1$

2、 $T(1) = 1$,

$$T(n) = \begin{cases} T(\frac{n}{2}) + 1, & n = 2^k, k \in \mathbb{N}_+ \\ 2T(\frac{n-1}{2}), & n = 2k+1, k \in \mathbb{N}_+ \end{cases}$$

(1) 证明 $n = 2^k$ 时, $T(n) = O(\log n)$

(2) 证明存在无穷集合 X , 满足 $n \in X$ 时 $T(n) = \Omega(n)$

(3) 由(1)(2)能得出什么结论?

解:

$$(1) T(2^k) = T(2^{k-1}) + 1 \therefore T(2^k) = k + 1$$

$$\therefore T(n) = k + 1 = \log_2 n + 1 = O(\log n)$$

$$(2) \text{ 令 } X = \{n | n = 2^k - 1, k \in \mathbb{N}_+\}$$

$$T(2^k - 1) = 2T(\frac{2^k - 1 - 1}{2}) = 2T(2^{k-1} - 1)$$

$$\therefore T(2^k - 1) = 2^{k-1}, T(n) = 2^{\log_2(n+1)-1} = \frac{n+1}{2} = \Omega(n)$$

(3) 为算法分析方便, 我们经常假设 $n = 2^k$, 并简单地加以推广, 但这并不总是正确的

3、设计算法求给定序列的最大连续子列积

解:

在已知已扫描过的部分的最大连续子列积、最大后缀子列积的基础上，还要保存并更新最小连续子列积，保证后续扫描到负数时的正确性

4、给定 `nums[1..n]` 表示股票在第1~n天的价格，若买入、卖出各1次，设计算法求最大收益

解法一：

申请的数组 `diff[n-1]`，`diff[i]` 保存第 `i+1` 天与第 `i` 天的价格差，在 `diff[n-1]` 的基础上求最大连续子列和

时间复杂度 $O(n)$

解法二：

申请2个数组 `A[n]`，`B[n]`

- `A[i]` 保存前 `i` 天的最低价格，可扫描 `nums[1..n]` 初始化 `A[]`
- `B[i]` 保存后 `n-i` 天的最低价格，可扫描 `nums[n..1]` 初始化 `B[]`

然后对 `i = 1 to n` 计算 `B[i] - A[i]` 的最大值

时间复杂度 $O(n)$

5、汉诺塔问题

(1)给出汉诺塔问题的移动算法

(2)已知各个圆盘状态（在给出的各个圆盘在柱子的分布中，小盘一定在大盘上方），求出这是(1)中算法的步数（若该状态在(1)中不会出现，则返回-1）

解：

(1)

```
//Algorithm: Hanoi
//Input: n, src, mid, dst
//Output: Each step
Hanoi(n, src, mid, dst)
{
    if(n==1) step = src + "->" + dst;
    else
    {
        Hanoi(n-1, src, dst, mid);
        step = src + "->" + dst;
        Hanoi(n-1, mid, src, dst);
    }
}
```

(2)从最大盘开始考虑

- 如果最大盘在 src ，则问题简化为：将 $1..n-1$ 从 src 移动到 mid 的过程，即求当前状态在 $Hanoi(n-1, src, dst, mid)$ 中的步数
- 如果最大盘在 dst ，则问题简化为：已经将 $1..n-1$ 从 src 移动到 mid ，共花费了 2^{n-1} 步，只需要求当前状态在 $Hanoi(n-1, mid, src, dst)$ 中的步数
- 如果最大盘在 mid ，则不可能出现该情况，返回-1

```
//Algorithm: # of steps in Hanoi
//Input: Condition
//Output: # of steps
Step(n, m, src, mid, dst)
{
    if (m == 0) return 0;
    if (n-m+1 at mid) return -1;
    if (n-m+1 at src) return Step(n, m-1, src, dst, mid);
    if (n-m+1 at dst)
    {
        tmp = Step(n, m-1, mid, src, dst);
        if (tmp == -1) return -1;
        else return 2^(m-1) + tmp;
    }
}
```

6、 $A[1..n]$ 为整数数组，构造 $B[1..n]$ ，其中 $B[i] = \max_{i \leq j \leq n} A[i]A[j]$ ，求构造 $B[1..n]$ 的算法及其时间复杂度

解：

扫描 $A[n..1]$ ，维护最大正数和最小负数，遇到 $A[i]$ 时根据 $A[i]$ 的正负情况，与前述两数之一相乘以构造 $B[i]$

时间复杂度 $O(n)$

Week 4

1、 给定单调递增数列，设计高效算法求出绝对值最小的元素

解：

- 若第1个数为正，则返回第1个数
- 若第n个数为负，则返回第n个数
- 否则二分查找，只需附加中间值为0直接返回的机制即可
- 时间复杂度 $O(\log n)$

2、给定数组 $A[1..n]$ ，相邻元素最多相差 1， $A[1] = x$ ， $A[n] = y$ ， $x \leq z \leq y$ ，求 z 在 $A[]$ 中出现的任意 1 个位置，设计算法并给出时间复杂度

解：

可类比数学中连续函数的性质和零点存在定理，二分查找保证找到 1 个解

时间复杂度 $O(\log n)$

3、我们知道，归并排序时间复杂度 $O(n \log n)$ ，插入排序时间复杂度 $O(n^2)$ ；但在数据规模足够小时，由于常数因素，插入排序比归并排序更快

考虑在数组被分割到足够小时，用插入代替归并以优化算法，对长度为 k 的 $\frac{n}{k}$ 个子序列使用归并合并

(1) 证明：对长度为 k 的 $\frac{n}{k}$ 个子序列使用插入排序的时间复杂度为 $O(nk)$

(2) 设计算法将子序列归并，要求时间复杂度 $O(n \log \frac{n}{k})$

(3) 由(1)(2)可知总复杂度 $O(nk + n \log \frac{n}{k})$ ，令 $O(nk + n \log \frac{n}{k}) = O(n \log n)$ 的最大 k 是多少？用 $O(f(n))$ 的形式表示

(4) 实际编程实现算法时，应如何选择 k ？

解：

(1)

- 单个序列 $O(k^2)$
- 总复杂度 $O(\frac{n}{k})O(k^2) = O(nk)$

(2)

- 成对地将子序列合并，每一级均为 $O(n)$ ，共 $\lceil \log \frac{n}{k} \rceil$ 级，总复杂度 $O(n \log \frac{n}{k})$
- 堆排序：维护最大二叉堆，堆中元素记录待排序的数据和数据所属的队列，每次出堆时，从出堆元素所属子列中选择下个元素入堆，堆操作 $\lceil \log \frac{n}{k} \rceil$ ，共 n 个元素，总复杂度 $O(n \log \frac{n}{k})$

(3) $k = O(\log n)$

- 显然 $k = O(1)$ 时 $n \log \frac{n}{k} = O(n \log n)$
- 若 $\log n = o(k)$ ，则 $n \log n = o(nk)$ ，不成立
- 故 k 的最高阶数为 $O(\log n)$

(4) k 应为插入排序比归并排序更快的最大数组规模

4、有 n 个不同大小的螺母和 n 个不同大小的螺栓，且能恰好完全配对。螺母和螺母不能比较大小，螺栓和螺栓不能比较大小。螺栓与螺母尝试匹配时，若松动，说明螺栓较小；若无法插入，说明螺母较小；否则说明螺母与螺栓恰好匹配。设计匹配全部螺母和螺栓的方法，并给出比较次数的期望阶数

解：

根据快速排序思想，用1个螺母与全部螺栓尝试匹配，可将螺母分为较小、较大、恰好匹配3组（其中恰好匹配只有1个）；再用恰好匹配的螺母与所有螺栓尝试匹配，可将螺栓分为较小、较大、恰好匹配3组（其中恰好匹配只有1个）；之后较小的螺母与较小的螺栓按同样方法匹配，较大的螺母与较大的螺栓匹配（递归）

比较次数 $T(n)$ 满足 $T(n) = \frac{\sum_{s=0}^{n-1} (T(s) + T(n-1-s))}{n} + 2n$ ，与快速排序类似，所以 $T(n) = O(n \log n)$

5、设 $A[1..n]$ 是整数 $1..n$ 的任意排列，若 $i < j$ 时 $A[i] > A[j]$ ，称 $\langle i, j \rangle$ 为 A 中的1个逆序对

(1) 列出 $A = \{2, 3, 5, 4, 1\}$ 中的逆序对

(2) $A[1..n]$ 最多包含多少个逆序对？此时 $A[1..n]$ 的数据分布情况如何？

(3) 设计算法求 $A[1..n]$ 中的逆序对数目，要求时间复杂度 $O(n \log n)$

解：

(1) $\langle 1, 5 \rangle, \langle 2, 5 \rangle, \langle 3, 5 \rangle, \langle 4, 5 \rangle, \langle 3, 4 \rangle$

注意：逆序对用其下标表示

(2) $\frac{n(n-1)}{2}$ 个； $A[1..n]=[n..1]$

(3) 根据归并排序思想，将数组元素从大到小排序

- size = 1 直接返回
- size = 2
 - 若 $A[i-1] < A[i]$ 则交换两元素
 - 否则不交换元素，但逆序对数 +1
- size >= 3 归并过程：前半数组中的元素出队时，逆序对数 + 后半数组中剩余元素的数量
- 时间复杂度：给归并排序每个 $O(1)$ 步骤附加了一个 $O(1)$ 步骤，最终复杂度 $O((1 + O(1))n \log n) = O(n \log n)$

6、证明如下排序算法的时间复杂性，并求其时间复杂度

```
//Algorithm: 3-sort
//Input: A[1..n]
//Output: A[1..n] in order
three_sort(Array[], left, right)
if (left + 1 == right)
    if (Array[left] > Array[right])
        swap(Array[left], Array[right])
if (left + 1 <= right)
    k = (right-left+1)/3
    three_sort(Array[], left, right-k)
    three_sort(Array[], left+k, right)
    three_sort(Array[], left, right-k)
```

解：

- 数组只有 1 个元素，则什么也不做，确保数组有序
- 数组只有 2 个元素，则条件交换，确保数组有序
- 数组包含 ≥ 3 个元素：
 - $\because 1 \leq k \leq \frac{right-left+1}{3}$
 $\therefore left < right - k, left + k < right$, 确保参数合法
 - 考虑最大的 $\lfloor \frac{n}{3} \rfloor$ 个元素
 - 若其初始位置位于最右侧的 $\lfloor \frac{n}{3} \rfloor$, 则第 1 轮 3-sort 不影响其位置
 - 否则, 第 1 轮 3-sort 后一定会出现在中间 $\lfloor \frac{n}{3} \rfloor$
 - 无论如何, 第 2 轮 3-sort 后一定会被放在最右侧的 $\lfloor \frac{n}{3} \rfloor$, 确保数组有序
 - 第 3 轮 3-sort 一定能将其余 $\lceil \frac{2n}{3} \rceil$ 排序
- 求时间复杂度：

$$T(n) = 3T\left(\frac{2n}{3}\right) + O(1)$$

$$T(n) = O(n^{\log_{\frac{3}{2}} 3})$$

- 优于 $O(n^2)$ 的排序算法如插入排序、冒泡排序，但劣于 $O(n \log n)$ 的排序算法如归并排序、快速排序